

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет прикладної математики та інформатики
(повне найменування назва факультету)

Кафедра інформаційних систем
(повна назва кафедри)

КВАЛІФІКАЦІЙНА (БАКАЛАВРСЬКА)
РОБОТА
РОЗРОБКА КЛІЄНТСЬКОЇ ЧАСТИНИ
МОБІЛЬНОГО ЗАСТОСУНКУ ДЛЯ ОБМІНУ
ПОДАРУНКАМИ «VAZNAU!»

Виконав(ла): студент(ка) групи ПМІ-44
спеціальності 122 – комп'ютерні науки
(шифр і назва спеціальності)

_____ Стегней В. І.
(підпис) (прізвище та ініціали)

Керівник _____ Горlach В.М.
(підпис) (прізвище та ініціали)

Рецензент _____
(підпис) (прізвище та ініціали)

	2
ВСТУП.....	5
1. ПОСТАНОВКА ЗАДАЧІ.....	7
2. МОДЕЛЬ ДАНИХ.....	9
2.1. Користувач (User).....	9
2.2. Бажання (Wish).....	10
2.3. Резервування (Reservation).....	11
2.4. Бренд (Brand).....	11
2.5. Новина (Article).....	11
2.6. Підписка (Subscription).....	12
2.7. Аналітика.....	12
3. ВИКОРИСТАНИЙ НАБІР ТЕХНОЛОГІЙ.....	13
3.1. Основні технології.....	13
3.1.1. React Native [1].....	13
3.1.2. Expo [2].....	13
3.1.3. TypeScript.....	13
3.1.4. Firebase [3].....	13
3.2. Додаткові бібліотеки.....	13
3.2.1. React Navigation [10].....	13
3.2.2. AsyncStorage [11].....	14
3.2.3. Formik [12] + Yup [13].....	14
3.2.4. React Native Fast Image [14].....	14
3.2.5. React Native Reanimated [15].....	14
3.3. Серверні взаємодії.....	14
3.3.1. Django REST Framework [4].....	14
3.3.2. PostgreSQL [5].....	14
3.3.3. Redis [6].....	14
4. ОСОБЛИВОСТІ ПРОГРАМНОЇ РЕАЛІЗАЦІЇ.....	15

4.1. Архітектура застосунку	15
4.2. Обробка стану	16
4.3. Навігація	16
4.3.1. Deep links	16
4.4. Інтеграція з сервером	16
4.4.1. Канал технічної підтримки	17
4.5. Аутентифікація та авторизація	17
4.6. Локалізація	18
4.7. Push-сповіщення	18
4.8. Кешування на стороні клієнту	18
4.9. Преміум-підписка та монетизація	19
4.10. Обробка мультимедійного контенту	19
4.11. Аналітика та моніторинг стабільності	20
4.12. Соціальні механіки	20
4.13. Безпека і приватність	21
4.14. Вибрані особливості	21
5. ОПИС ІНТЕРФЕЙСУ КОРИСТУВАЧА	22
5.1. Загальні принципи дизайну	22
5.2. Основні екрани	22
5.2.1. Вітальні екрани	22
5.2.2. Екран реєстрації та входу	23
5.2.3. Головна стрічка	24
5.2.4. Сторінки брендів, новин та бажань	24
5.2.5. Сторінка пошуку	25
5.2.6. Сторінка “Ком’юніті”	26

5.2.7. Створення бажання	26
5.2.8. Профіль користувача	29
5.2.9. Сторінка налаштувань	30
5.2.10. Сторінка редагування профілю.....	30
5.2.11. Сторінки налаштування мови та преміум підписки	31
5.2.12. Сторінки поширених питань та технічної підтримки.....	32
5.2.13. Сторінка видалення профілю	32
5.3. Додаткові елементи	34
5.4. Платформи	36
5.4.1. Конфігурація платформ	36
5.4.2. Публікація та оновлення.....	37
6. ОТРИМАНІ РЕЗУЛЬТАТИ ТА МОЖЛИВОСТІ ДЛЯ ПОДАЛЬШОГО РОЗВИТКУ.....	38
6.1. Отримані результати	38
6.2. Новизна роботи.....	39
6.3. Перспективи подальшого розвитку	39
ВИСНОВКИ	40
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	42

ВСТУП

У сучасному цифровому світі все більшої популярності набувають сервіси, що покликані полегшити повсякденні процеси, пов'язані з комунікацією, вибором подарунків та взаємодією між людьми. Однією з поширених проблем, з якою стикаються користувачі, є складність у виборі відповідного подарунка для близьких. Через це дарувальники часто обирають речі, які не приносять радості одержувачам, а сам процес дарування втрачає свою емоційну цінність. Актуальність вирішення цієї проблеми зумовила створення мобільного додатку «BAZHAY!».

«BAZHAY!» — це мобільний застосунок, що дозволяє користувачам публікувати власні бажання та списки бажань (вішлисти), надаючи друзям, рідним і близьким можливість резервувати чи дарувати ці бажання. Цільова аудиторія додатку — люди різного віку, які прагнуть уникнути небажаних подарунків і зберегти приємні емоції у святкові моменти. Додаток також передбачає функції копіювання бажань, перегляду стрічки рекомендацій, а головною його особливістю є універсальний парсер сайтів, що дозволяє створювати бажання швидко та зручно, імпортуючи інформацію безпосередньо з інтернет-магазинів.

Клієнтська частина застосунку реалізована за допомогою фреймворку React Native [1] у середовищі Expo [2], що дозволяє забезпечити кросплатформенність і високу продуктивність. Для аналітики використовується Firebase [3]. Серверна частина, хоч і не є предметом розгляду в даній курсовій роботі, побудована на основі Django REST Framework [4] з використанням бази даних PostgreSQL [5] та кешування через Redis [6]. Також у застосунку реалізовано можливість рекламування брендів, що відкриває перспективи для монетизації.

Метою даної бакалаврської роботи є розробка клієнтської частини мобільного додатку «BAZHAY!», включно з архітектурним проєктуванням, реалізацією основних функціональних модулів, проведенням тестування та публікацією готового продукту в App Store [7] та Google Play [8].

У ході роботи були використані такі методи: аналіз потреб користувачів, проектування архітектури застосунку, реалізація інтерфейсу та логіки клієнтської частини, ручне тестування функціональності.

Бакалаврська робота складається з кількох розділів, у яких розглянуто загальну концепцію проєкту, архітектурні рішення клієнтської частини, особливості розробки ключових функцій, результати реалізації та подальші перспективи розвитку. Проєкт уже доступний користувачам на мобільних платформах і активно використовується за призначенням — для створення та здійснення бажань, що наближає його до реального впровадження в повсякденне життя.

1. ПОСТАНОВКА ЗАДАЧІ

У сучасному суспільстві значну роль відіграє культура дарування подарунків, однак часто виникає проблема вибору відповідного подарунка, що дійсно відповідає бажанням одержувача. Через відсутність ефективного засобу для комунікації бажань та очікувань між дарувальником і одержувачем, процес підбору подарунків супроводжується невизначеністю, стресом та часто не виправдовує очікувань обох сторін.

Метою даної роботи є розробка клієнтської частини мобільного застосунку «BAZHAY!», який забезпечує можливість формування персонального списку бажань користувача (вішлисту) та дозволяє іншим користувачам резервувати бажання для подальшого їх дарування.

Завдання, що необхідно виконати для досягнення поставленої мети:

- забезпечити створення бажань користувачем, включаючи автоматичне заповнення інформації за допомогою універсального парсера товарів з сайтів;
- реалізувати стрічку бажань та рекомендацій для перегляду нових ідей;
- передбачити можливість резервування бажань іншими користувачами;
- реалізувати функцію копіювання бажань з чужих вішлистів до власного;
- створити інтерфейс, зручний для взаємодії з додатком на різних платформах (iOS та Android);
- забезпечити публікацію додатку в App Store [7] та Google Play [8];
- спроектувати архітектуру клієнтської частини, яка дозволить масштабувати функціонал у майбутньому (наприклад, додавання чатів між користувачами).

До основних вимог до клієнтського застосунку належать:

- адаптивність і кросплатформеність;
- простота та інтуїтивність користувацького інтерфейсу;

- можливість швидкого створення бажань з мінімальною кількістю кроків;
- інтеграція з аналітичними сервісами для збору статистики;
- підтримка динамічного оновлення контенту без потреби в оновленні додатку.

У результаті виконання даної роботи має бути реалізований повнофункціональний мобільний застосунок, який вирішує поставлену проблему та відповідає зазначеним вимогам і може бути використаний широким колом користувачів для планування та здійснення бажань.

2. МОДЕЛЬ ДАНИХ

У рамках клієнтської частини мобільного додатку «BAZHAY!» використовується спрощене, але достатньо функціональне представлення даних, яке дозволяє зручно відображати, зберігати у локальному стані та надсилати інформацію до серверної частини.

Основними сутностями клієнтської моделі є:

2.1. Користувач (User)

Кожен користувач має обліковий запис, пов'язаний з певними персональними даними. У клієнтському застосунку зберігаються такі атрибути користувача:

- id: унікальний ідентифікатор користувача;
- email: електронна пошта;
- username: нікнейм користувача;
- first_name: ім'я користувача;
- las_name: прізвище користувача;
- birthday: дата народження користувача;
- view_birthday: змінна, що позначає чи інші користувачі можуть бачити рік народження користувача;
- sex: стать користувача;
- photo: посилання на зображення профілю;
- about_user: короткий опис користувача;
- subscriber, subscription: кількість підписників та підписок користувача;
- is_premium: чи має користувач преміум;
- is_unviewed_notification: чи має користувач непрочитані повідомлення;
- is_already_registered: чи користувач повністю заповнив свої дані під час реєстрації;

2.2. Бажання (Wish)

Бажання — ключова одиниця даних, яка відображає те, що користувач хоче отримати як подарунок. Основні поля:

- `id`: унікальний ідентифікатор бажання;
- `name`: назва бажання;
- `price`: ціна бажання;
- `link`: посилання на бажання;
- `currency`: валюта, в якій вказано ціну бажання;
- `description`: опис бажання;
- `access_type`: тип доступу до бажання (для всіх, для підписників, для вибраних користувачів, для себе);
- `photo`: посилання на фото бажання;
- `video`: посилання на відео бажання;
- `is_fully_created`: змінна, що позначає чи бажання створене до кінця, чи воно є чернеткою;
- `author`: користувач, що створив бажання;
- `image_size`: розмір фото у співвідношенні, наприклад 9:16;
- `is_reservation`: чи зарезервоване це бажання;
- `is_your_wish`: чи це бажання користувача, що зараз переглядає його;
- `is_reserved_by_me`: чи бажання зарезервоване користувачем, що зараз переглядає його;
- `brand_author`: автор бажання, якщо воно створене брендом;
- `is_fulfilled`: чи виконане це бажання;
- `is_me_candidates_to_reservation`: чи користувач, що зараз переглядає це бажання, є кандидатом на його резервацію;
- `is_copied_from_brand`: чи бажання скопійоване з бажання бренду;

Бажання може створюватися вручну або автоматично за допомогою універсального парсера – за URL системний модуль завантажує дані про товар (назву, фото, ціну тощо) і формує бажання.

2.3. Резервування (Reservation)

Функціонал резервування дозволяє іншим користувачам зарезервувати бажання, щоб уникнути дублювання подарунків. Для його реалізації використано таку модель даних:

- id: унікальний ідентифікатор резерву;
- selected_user: користувач, який зарезервував бажання;
- wish_id: ідентифікатор бажання;
- candidates: користувачі, кандидати на резервування;

2.4. Бренд (Brand)

Бренди – одна з особливостей додатку. Завдяки системі брендів, «BAZHAY!» має потенціал на монетизацію. Основні поля:

- slug: унікальний текстовий ідентифікатор, також відомий як “слаг”;
- name: назва бренду;
- nickname: ім’я бренду, аналогія до поля username в моделі User;
- photo: посилання на фото бренду;
- description: опис бренду;
- cover_photo: посилання на зображення для фону сторінки бренду;

2.5. Новина (Article)

Новини відображаються на головній сторінці додатку та відображають головні новини додатку. Основні поля:

- slug: унікальний текстовий ідентифікатор;
- title: заголовок новини;
- photo: посилання на зображення новини;
- description: основне наповнення новини;

2.6. Підписка (Subscription)

Підписка слугує для відображення того, хто на кого з користувачів підписувався. Поля:

- id: унікальний ідентифікатор підписки;
- user: користувач, який підписався;
- subscribed_to: користувач, на якого підписались;

2.7. Аналітика

На рівні клієнта також підтримується передача подій (аналітика Firebase [9]), наприклад:

- wish_created;
- screen_view;
- wish_fulfilled;
- in_app_purchase тощо.

3. ВИКОРИСТАНИЙ НАБІР ТЕХНОЛОГІЙ

У процесі розробки клієнтської частини мобільного застосунку «**VAZHAY!**» було використано сучасні фреймворки, бібліотеки та інструменти, що забезпечують швидку, зручну й кросплатформенну розробку.

3.1. Основні технології

3.1.1. React Native [1]

React Native [1] є основною технологією для побудови інтерфейсу користувача. React Native [1] дозволяє створювати кросплатформені мобільні застосунки на базі JavaScript/TypeScript з єдиною кодовою базою для iOS та Android.

3.1.2. Expo [2]

Expo [2] є фреймворком, що спрощує розробку на базі React Native [1]. Завдяки Expo [2] значно зменшено час налаштування середовища та компіляції, а також спрощено доступ до API пристрою (камера, галерея, push-нотифікації тощо).

3.1.3. TypeScript

TypeScript – це типізована надбудова над JavaScript, що підвищує надійність і стабільність коду за рахунок статичної типізації. У проєкті ця технологія використовується для структурованості та зменшення помилок.

3.1.4. Firebase [3]

Firebase [3] – це сервіс Google, який у проєкті використовується для:

- збору аналітики користувацької поведінки (Firebase Analytics [9]);
- тестування оновлень та поведінки користувачів у реальному часі.

3.2. Додаткові бібліотеки

3.2.1. React Navigation [10]

Для організації навігації між екранами застосунку було використано бібліотеку React Navigation [10]. Зокрема, використовується стекова та табова навігація.

3.2.2. AsyncStorage [11]

Бібліотека AsyncStorage [11] використовується для збереження токена авторизації та інших ключових налаштувань користувача на пристрої, а також для кешування з клієнтської сторони.

3.2.3. Formik [12] + Yup [13]

Форми з валідацією – важлива складова кожного додатку, тож для їх реалізації використано поєднання бібліотек Formik [12] + Yup [13] і використано для реєстрації, створення бажань, налаштувань профілю тощо.

3.2.4. React Native Fast Image [14]

React Native Fast Image [14] використовується для швидкого вивантаження фото з серверу та їх відображення на екрані. Бібліотека надає кешування зображень на стороні клієнту.

3.2.5. React Native Reanimated [15]

Анімації сприяють кращому користувацькому досвіду, тому вони теж реалізовані в додатку з використанням бібліотеки React Native Reanimated [15], в особливості для відображення іконки загрузки та анімації відкриття додатку.

3.3. Серверні взаємодії

3.3.1. Django REST Framework [4]

Серверна частина реалізована на Django REST Framework (DRF) [4], з яким взаємодіє клієнтська частина. Всі дані – бажання, користувачі, резервування – отримуються через REST API.

3.3.2. PostgreSQL [5]

Для зберігання інформації про користувачів, бажання, резервування, тощо, використано реляційну базу даних PostgreSQL [5].

3.3.3. Redis [6]

Redis [6] використовується для кешування, оптимізації рекомендацій та прискорення обробки запитів.

4. ОСОБЛИВОСТІ ПРОГРАМНОЇ РЕАЛІЗАЦІЇ

Проект клієнтської частини мобільного додатку «BAZHAY!» реалізовано з урахуванням вимог до масштабованості, зручності підтримки, розширюваності та швидкодії. У цьому розділі буде розглянуто структуру застосунку, архітектурні рішення, підхід до навігації та зберігання стану.

4.1. Архітектура застосунку

Клієнтська частина додатку реалізована відповідно до компонентно-модульного підходу. Основні модулі:

assets – папка, в якій зберігаються такі файли додатку, як зображення, шрифти, а також конфігураційні файли для Firebase Analytics [9];

components – спільні компоненти (UI), такі як відображення тексту, поля вводу, картки бажань і т.д.;

contexts – React Contexts [16], що використовуються в додатку, наприклад: LocalizationContext, AuthContext і т.п.;

locals – зібрані файли локалізації для впровадження української та англійської мов в додатку;

screens – всі сторінки додатку. Всередині сторінки також розбиті на модулі:

- **auth** – сторінки пов'язані з реєстрацією та логіном;
- **home** – сторінки, доступ до яких здійснюється з головної сторінки, наприклад сторінка новини, бренду чи сповіщень;
- **main** – основні сторінки з Tab-навігацією;
- **profile** – сторінки, пов'язані з профілем користувача;
- **welcome** – вітальні сторінки, що показуються при першому вході в додаток;
- **wishCreating** – блок сторінок, що слугують для створення бажання;

У кожному з цих модулів також реалізовано сервіси та dao-сервіси. Сервіси – це класи, в яких відбувається основна логіка додатку, а dao-сервіси – це класи, що слугують для відправки та отримання даних з серверу.

utils – збірка допоміжних функцій та утиліт.

Таке розділення дозволяє ізолювати логіку, повторно використовувати компоненти та полегшує подальшу підтримку проєкту.

4.2. Обробка стану

Для керування глобальним станом використовується **React Context API** [16]. Цей менеджер стану дозволяє зберігати інформацію про користувача та синхронізувати стан між екранами.

4.3. Навігація

Застосовано **React Navigation** [10], що дозволило реалізувати декілька типів навігації:

- **Stack-навігація** для базових переходів між екранами;
- **Tab-навігація** для основних розділів (профіль, стрічка, пошук);

4.3.1. Deep links

Навігаційна підсистема доповнена механізмом deep linking, який дозволяє відкривати конкретні екрани застосунку без проходження всього маршруту вручну. Підтримуються як внутрішня схема застосунку, так і HTTPS-посилання з перенаправленням через веб-домен BAZHAY!, що особливо важливо для поширення профілів, брендів, новин і окремих бажань.

Завдяки такій конфігурації користувач може перейти за зовнішнім посиланням і відразу потрапити на екран профілю, стрічки, бренду, новини, сторінки сповіщень або конкретного бажання. Це покращує інтеграцію мобільного застосунку з зовнішніми каналами комунікації та підсилює сценарії органічного поширення контенту.

4.4. Інтеграція з сервером

Всі дані отримуються через **REST API**, який надає серверна частина на Django REST Framework. Запити виконуються через **fetch** та **XMLHttpRequest** [17]. Для уніфікації HTTP-взаємодії в проєкті реалізовано допоміжні функції **fetchWithAuth** та **xhrWithAuth**. Перша використовується для більшості стандартних запитів до REST API: вона автоматично додає access-токен у заголовок

Authorization, а при отриманні відповіді зі статусом 401 викликає механізм оновлення токена і повторює запит без участі користувача.

Функція `xhrWithAuth` застосовується у сценаріях, де потрібно показати хід передавання великих даних, зокрема під час завантаження фото, відео чи вкладень до технічної підтримки. Вона побудована на `XMLHttpRequest` і дозволяє відстежувати прогрес надсилання та отримання файлів, що значно покращує сприйняття довготривалих операцій у мобільному інтерфейсі.

Усі запити максимально оптимізовані – наприклад, стрічка бажань використовує пагінацію (`cursor-based`), кешується та передає лише необхідні дані.

4.4.1. Канал технічної підтримки

Окремим сценарієм серверної взаємодії є технічна підтримка користувачів. Для цього в застосунку поєднано два способи обміну даними: швидке надсилання текстових повідомлень через **WebSocket** та передавання вкладень через HTTP `multipart`-запити.

Користувач може звернутися до підтримки як у вигляді звичайного текстового повідомлення, так і з доданим файлом із галереї чи камери. На клієнті передбачено перевірку розміру вкладення, щоб не допустити надсилання надто великих файлів, а структура окремих екранів підтримки спрощує опис помилки та подальший діалог із менеджером застосунку.

4.5. Аутентифікація та авторизація

Підсистема автентифікації підтримує декілька сценаріїв входу: підтвердження електронної пошти одноразовим кодом, вхід через Google, вхід через Apple та гостьовий режим. Такий підхід дає змогу не втрачати користувачів на старті, але водночас зберігає повноцінний сценарій персоналізації профілю для зареєстрованих користувачів.

Після успішного входу клієнт зберігає пару `access/refresh` токенів у локальному сховищі, а при отриманні відповіді зі статусом 401 автоматично виконує оновлення `access`-токена. Додатково реалізовано поетапне завершення реєстрації: користувач послідовно заповнює ім'я, нікнейм, дату народження, стать,

аватар та інші поля профілю, а застосунок запам'ятовує, на якому кроці цей процес було зупинено, щоб повернутись до нього, якщо користувач завчасно зупинить додаток.

4.6. Локалізація

Підтримка багатомовності побудована на основі окремого `LocalizationContext`, який централізовано зберігає поточну мову інтерфейсу та набір локалізованих текстів. Під час першого запуску застосунок зчитує системну мову пристрою, після чого обирає українську або англійську мову та зберігає вибір користувача в локальному сховищі.

Окремою перевагою такого підходу є те, що мова використовується не лише для відображення статичного інтерфейсу, але й для синхронізації суміжних підсистем. Зокрема, при зміні локалізації оновлюється мова підписки на push-сповіщення, що забезпечує узгоджений користувацький досвід у всіх каналах взаємодії.

4.7. Push-сповіщення

Для роботи зі сповіщеннями застосовано `Expo Notifications`. На клієнті реалізовано повний цикл: запит дозволу від користувача, отримання `Expo Push Token`, збереження токена на пристрої та передача цього токена на сервер разом із мовою інтерфейсу.

Отримані сповіщення не лише інформують користувача про нові події, а й впливають на локальний стан застосунку. Після надходження сповіщення очищується кеш стрічки та окремих бажань, а при відкритті сповіщення користувач перенаправляється на потрібний екран: список сповіщень, сторінку бажання або канал технічної підтримки.

4.8. Кешування на стороні клієнту

Кешування реалізовано за допомогою окремого сервісу, який має декілька методів: закешувати дані, видалити дані, дістати дані та очистити кеш. Такий підхід дозволив використати сервіс кешування у всіх інших сервісах, що стало універсальним рішенням в додатку.

Кешувальний сервіс побудовано поверх AsyncStorage, але його роль у проєкті значно ширша за звичайне локальне збереження довільних значень. Кожен запис містить не лише корисні дані, а й час завершення актуальності, тобто TTL. Це дозволяє автоматично вважати застарілими ті сутності, які не були оновлені у визначений проміжок часу, та уникати використання неактуальної інформації без ускладнення логіки окремих екранів.

Сервіс кешує як окремі сутності, так і колекції: профіль поточного користувача, профілі інших користувачів, списки бажань, результати пошуку, стрічку рекомендацій, бренди, новини, підписки, преміум-стан, адреси та поштові дані. Інвалідація кешу відбувається після мутацій, пов'язаних зі створенням, оновленням або видаленням об'єкта, а також після отримання push-сповіщень. Завдяки цьому користувач отримує швидкий інтерфейс, але при цьому не бачить застарілого контенту після важливих змін.

4.9. Преміум-підписка та монетизація

Монетизаційний блок застосунку побудовано на основі бібліотеки **react-native-iap**. Клієнт отримує актуальні ціни підписок із магазинів застосунків, окремо обробляє сценарії для iOS та Android і підтримує як щомісячну, так і річну модель доступу.

Після оформлення покупки клієнт передає на сервер необхідні підтверджувальні дані: чек iOS або purchase token Android. Також реалізовано активацію вже доступної підписки при повторному вході користувача в застосунок, що особливо важливо після перевстановлення програми або переходу на інший пристрій.

4.10. Обробка мультимедійного контенту

Створення бажання в BAZHAY! не обмежується лише текстовим описом. Користувач може додати фотографію або відео з камери чи галереї, після чого виконати додаткову обробку матеріалу без виходу із застосунку.

Для зображень реалізовано обрізання та масштабування під потрібне співвідношення сторін. Для відео підтримуються вибір фрагмента, попередній

перегляд кадрів, формування обкладинки та окреме завантаження відео й згенерованого cover-зображення із відображенням прогресу. Додатково передбачено копіювання медіафайлів у тимчасове сховище пристрою, якщо бажання створюється на основі вже існуючого запису.

4.11. Аналітика та моніторинг стабільності

Для супроводу та подальшого розвитку продукту в застосунку інтегровано Firebase Analytics та Firebase Crashlytics. Аналітичний шар фіксує як базові сценарії використання, так і більш прикладні бізнес-події: входи користувачів, перегляд окремих екранів, створення бажань, взаємодію з рекламними елементами, виконання бажань та інші дії, що характеризують поведінку аудиторії.

Окрему роль відіграє моніторинг стабільності. Crashlytics дозволяє збирати інформацію про аварійні завершення застосунку та контекст їх виникнення, а screen_view події допомагають зрозуміти, на яких екранах користувачі перебувають найчастіше. У сукупності це дає змогу приймати рішення не лише на основі суб'єктивних відгуків, а й на базі реальної статистики експлуатації продукту.

4.12. Соціальні механіки

Важливою складовою BAZHAY! є соціальні механіки, які виходять за межі звичайного списку бажань. Застосунок підтримує систему підписок між користувачами, перегляд підписників і підписок у розділі ком'юніті, копіювання бажань інших користувачів або брендів до власного профілю, а також нативний шаринг профілів, брендів і окремих сторінок через стандартні можливості мобільної операційної системи.

Додатковим елементом довіри є механізм запиту адреси або поштових даних іншого користувача лише за потреби. Це дозволяє зберегти приватність власника профілю, але водночас спростити реальне дарування подарунків. Саме поєднання підписок, обміну контентом, резервувань і контрольованого доступу до контактних даних формує в застосунку соціальне середовище, а не просто каталог побажань.

4.13. Безпека і приватність

Питання безпеки і приватності враховано на кількох рівнях клієнтської архітектури. Авторизаційні токени зберігаються локально та використовуються централізовано через допоміжні функції мережевої взаємодії, що зменшує ризик помилкового дублювання логіки доступу. При виході з облікового запису клієнт не лише очищує локальні дані, а й відкликає push-токен на сервері, запобігаючи подальшому надсиланню персоналізованих повідомлень на чужий пристрій.

Приватність користувача також захищено на рівні предметної логіки. Бажання можуть мати різні режими видимості: для всіх, для підписників, для вибраних користувачів або лише для власника. Доступ до адреси чи поштових даних не є автоматичним і надається окремо. Дозволи на камеру, галерею та роботу з медіафайлами застосунок запитує лише в тих сценаріях, де вони справді потрібні, що відповідає принципу мінімально необхідного доступу.

4.14. Вибрані особливості

Універсальний парсер – дозволяє користувачу вставити посилання на товар (наприклад, з Rozetka чи Amazon), і застосунок автоматично заповнює інформацію про бажання.

Копіювання бажання – користувач може скопіювати бажання іншого до свого списку – популярна функція серед користувачів.

Резервування – бажання показується користувачам як зарезервоване, щоб уникнути повторних подарунків.

Можливість додати бренд – бренди можуть створювати власні бажання, що дозволяє їм просувати товари в додатку.

5. ОПИС ІНТЕРФЕЙСУ КОРИСТУВАЧА

Інтерфейс користувача мобільного додатку «**BAZHAY!**» розроблений з урахуванням принципів зручності, інтуїтивності та сучасного дизайну. Усі екрани мають єдиний візуальний стиль, логічну структуру та адаптивність до різних розмірів екранів.

5.1. Загальні принципи дизайну

Мінімалізм – інтерфейс не перевантажений, основна увага на контенті (бажаннях).

Фокус на емоції – дизайн спрямований на створення позитивного досвіду від дарування та отримання подарунків.

Уніфіковані компоненти – кнопки, поля, заголовки мають узгоджений вигляд, що забезпечує зручність сприйняття.

5.2. Основні екрани

5.2.1. Вітальні екрани

Вітальні екрани призначені для ознайомлення нового користувача з додатком.

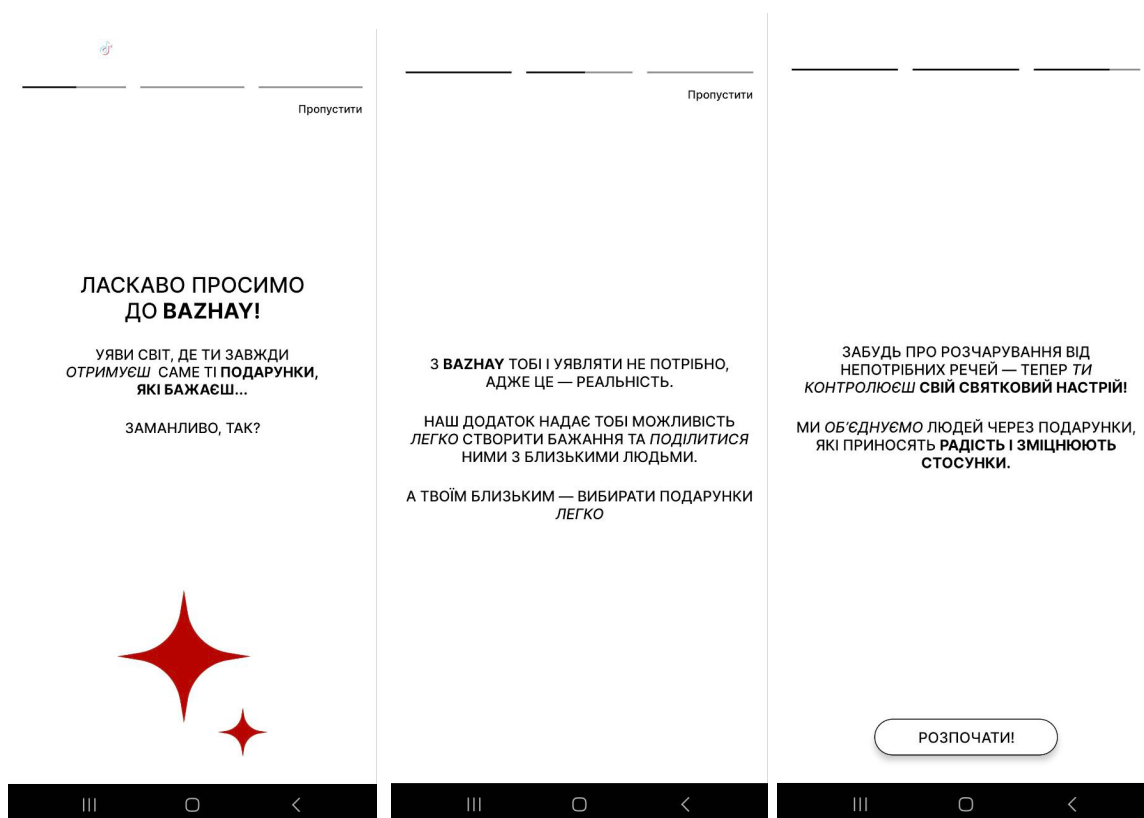


Рис. 5.1 – Вітальні екрани

5.2.2. Екран реєстрації та входу

Екран реєстрації та входу має простий і зрозумілий інтерфейс з валідацією даних. Наявна підтримка надсилання коду підтвердження. Після входу користувач потрапляє на головну стрічку. Якщо ж користувач ще не зареєстрований, його переадресовує на сторінки реєстрації.

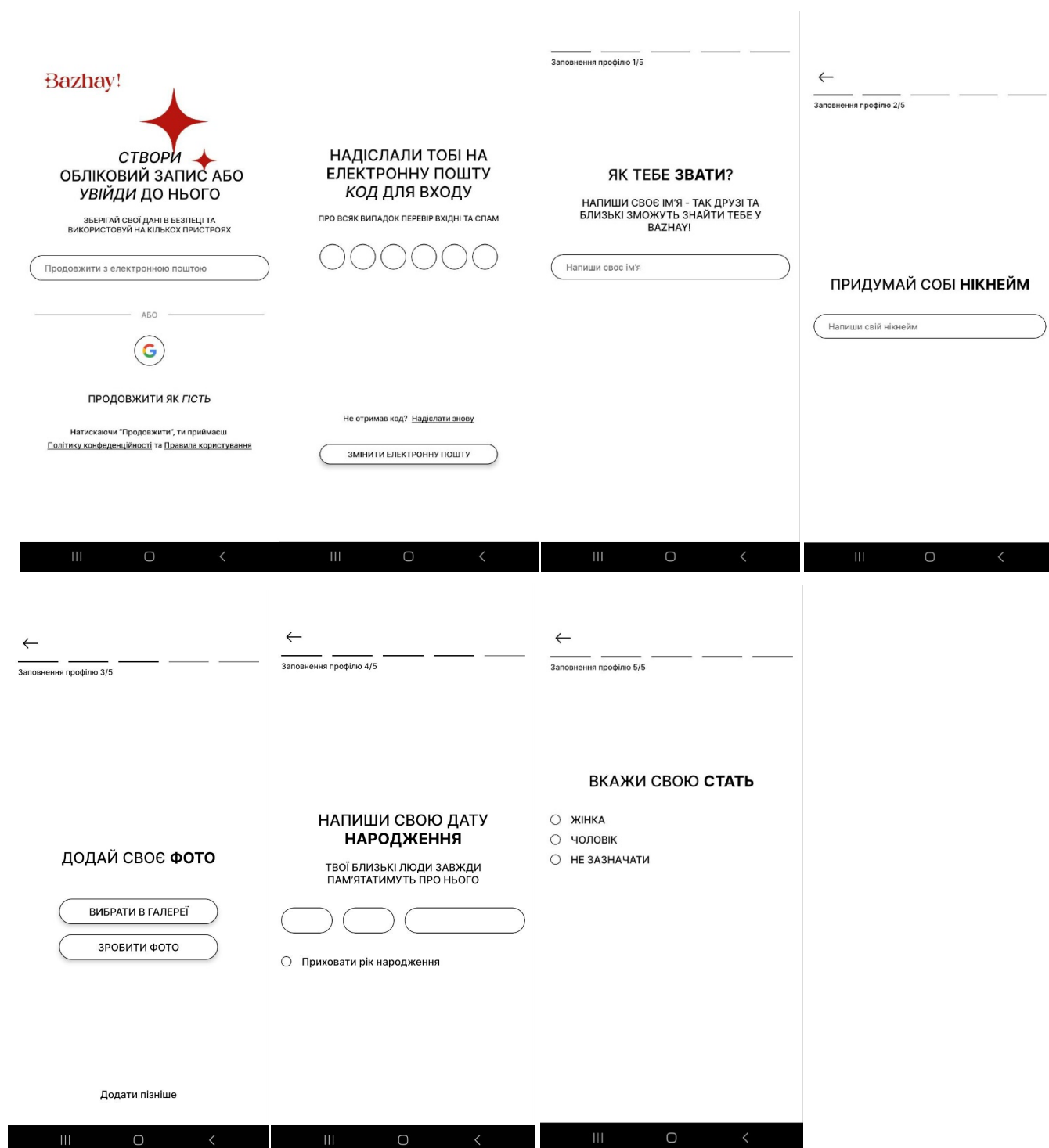


Рис. 5.2 – Екрани реєстрації та входу

Окрім базового входу через електронну пошту, інтерфейс цього блоку підтримує вхід через Google, Apple та гостьовий режим. Це дозволяє швидко залучити користувача до взаємодії із сервісом, а вже після першого входу провести його через послідовність екранів завершення профілю.

5.2.3. Головна стрічка

Головна стрічка відображає бажання інших користувачів із рекомендацій. Можливість переглядати, копіювати або резервувати бажання. Вивід всіх брендів, що співпрацюють з додатком, а також новин.

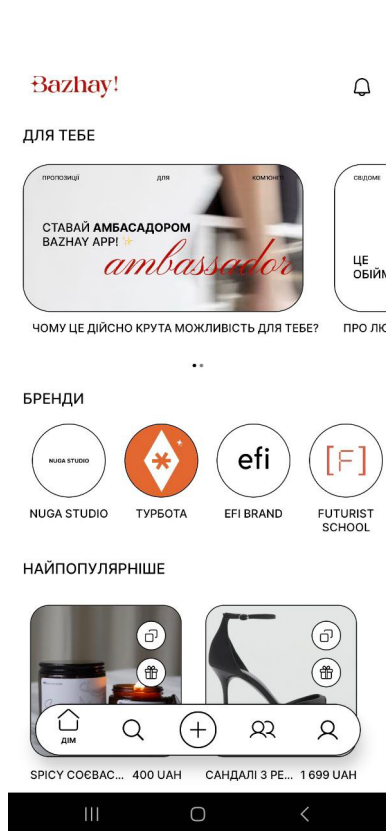


Рис. 5.3 – Головна сторінка

5.2.4. Сторінки брендів, новин та бажань

Сторінки брендів, новин та бажань призначені для відображення повної інформації про відповідні об'єкти. А також для резервування, копіювання та можливості поділитись бажаннями.

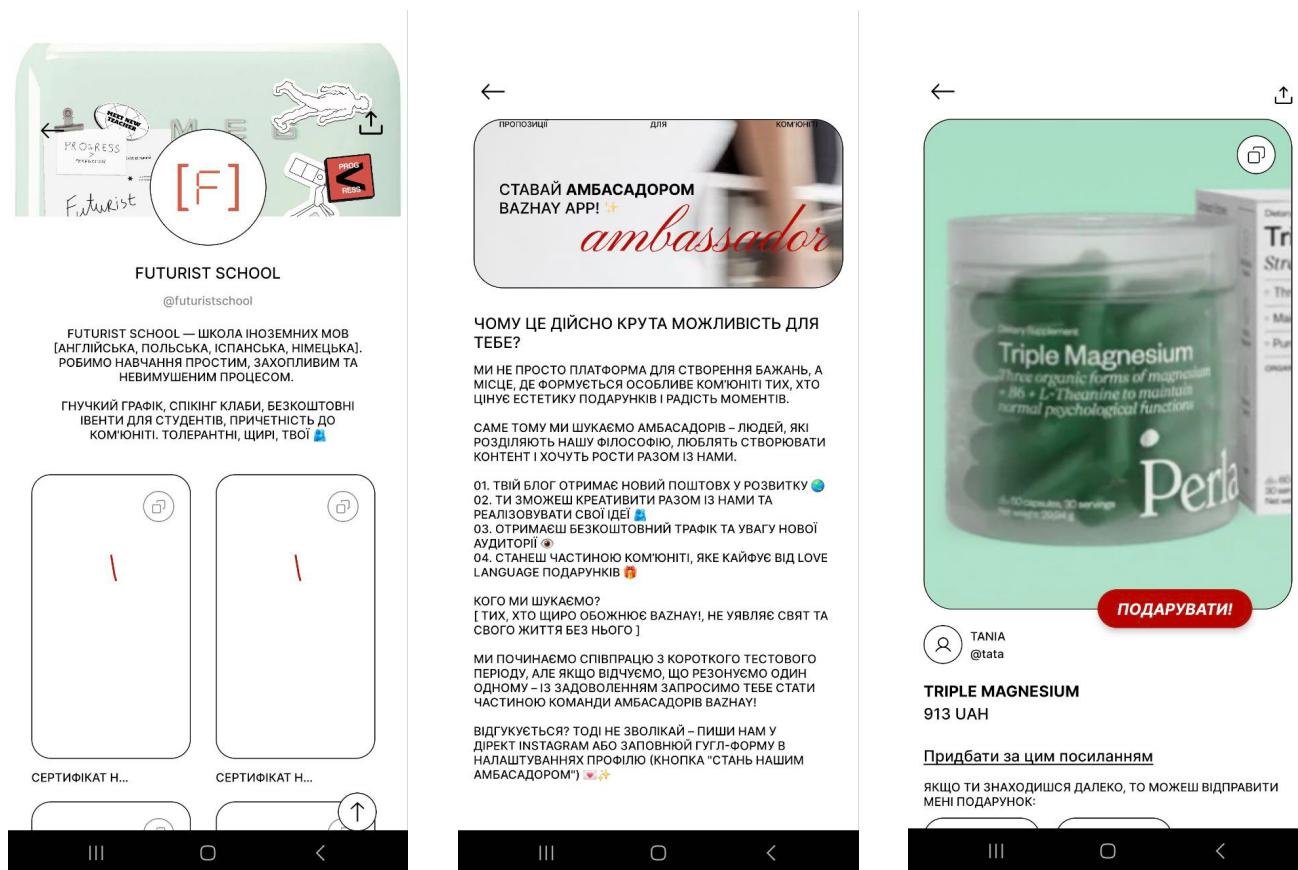


Рис. 5.4 – Сторінки бренду, новини та бажання відповідно

5.2.5. Сторінка пошуку

Сторінка пошуку дає змогу знаходити популярні запити, користувачів, бренди та бажання.

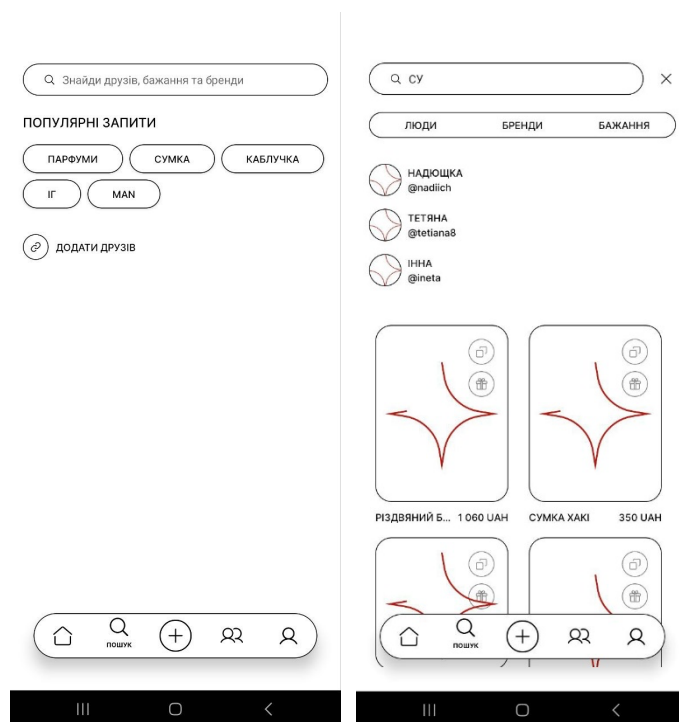


Рис. 5.5 – Сторінка пошуку

5.2.6. Сторінка “Ком’юніті”

Сторінка “Ком’юніті” надає можливість переглядати своїх підписників і підписки, а також знаходити близьких.

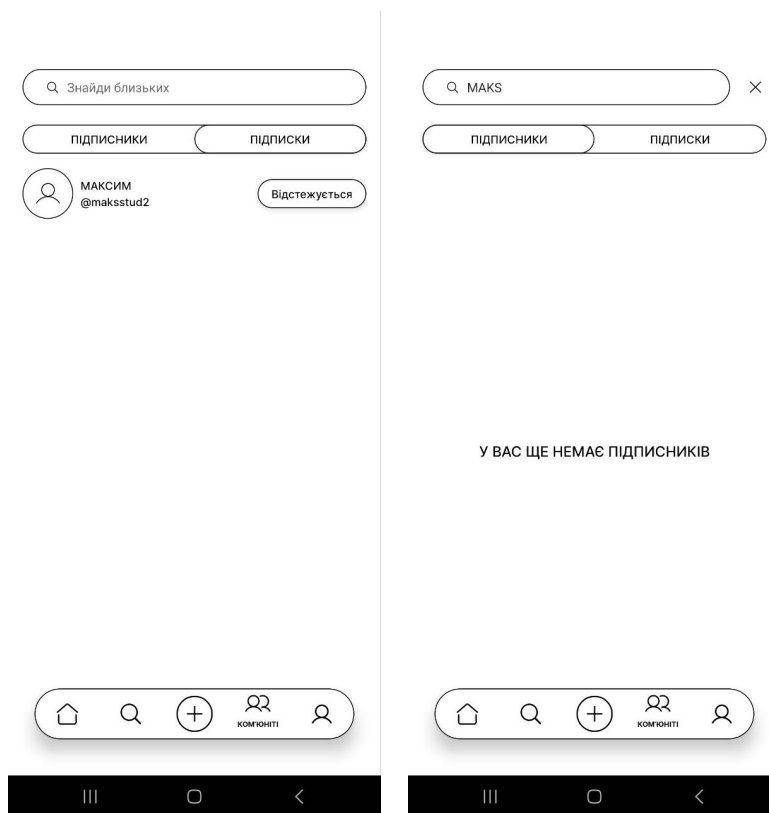


Рис. 5.6. – Сторінка “Ком’юніті”

5.2.7. Створення бажання

Поля для назви, опису, ціни, посилання, фото бажання, розділені на окремі сторінки для зручного заповнення даних. Інтегрований парсер сайтів – автозаповнення даних із посилання. Також є можливість переглянути заповненні дані, а також обрізати фото до потрібного розміру.

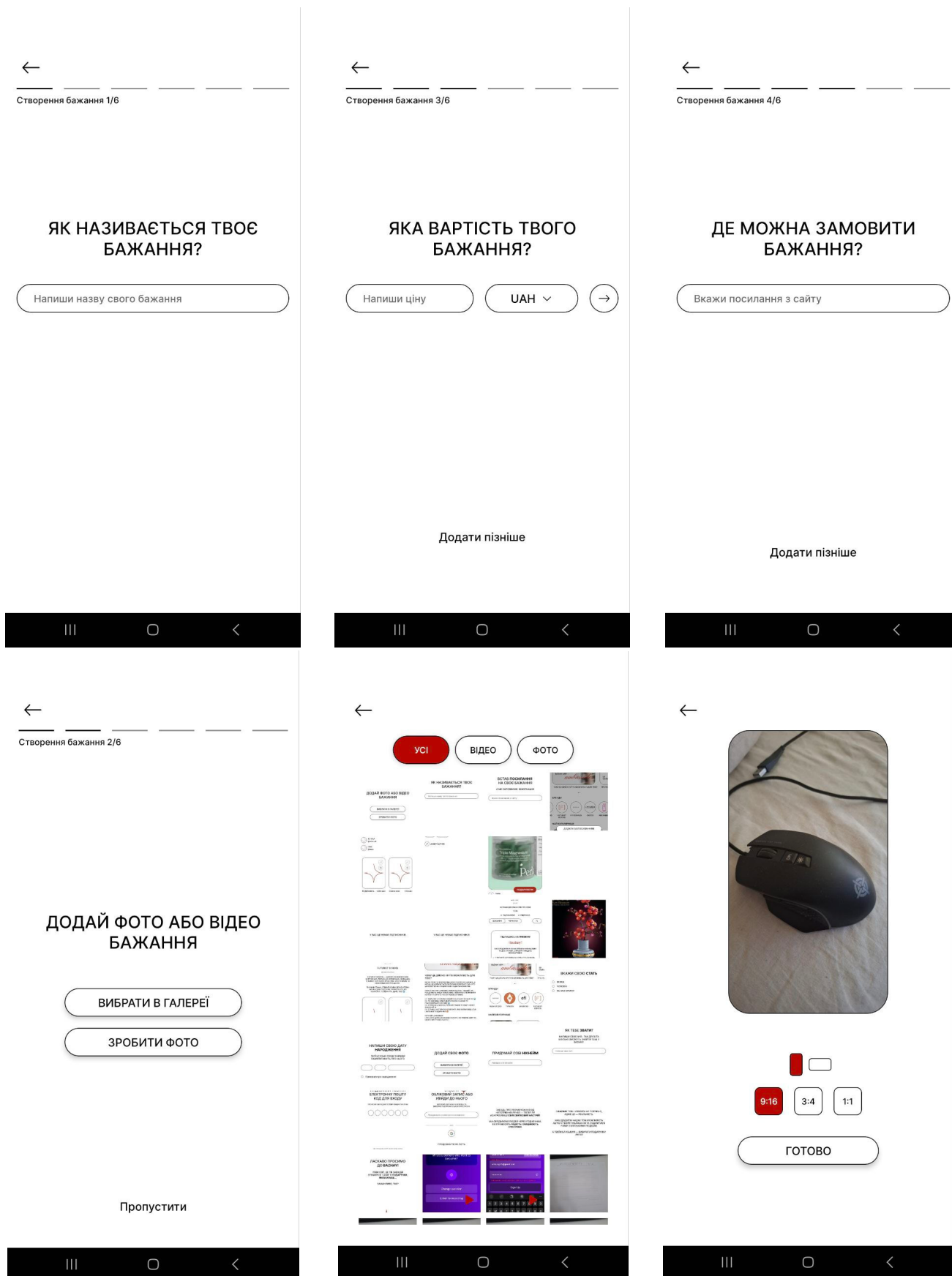


Рис. 5.7 – Екрани заповнення інформації про бажання

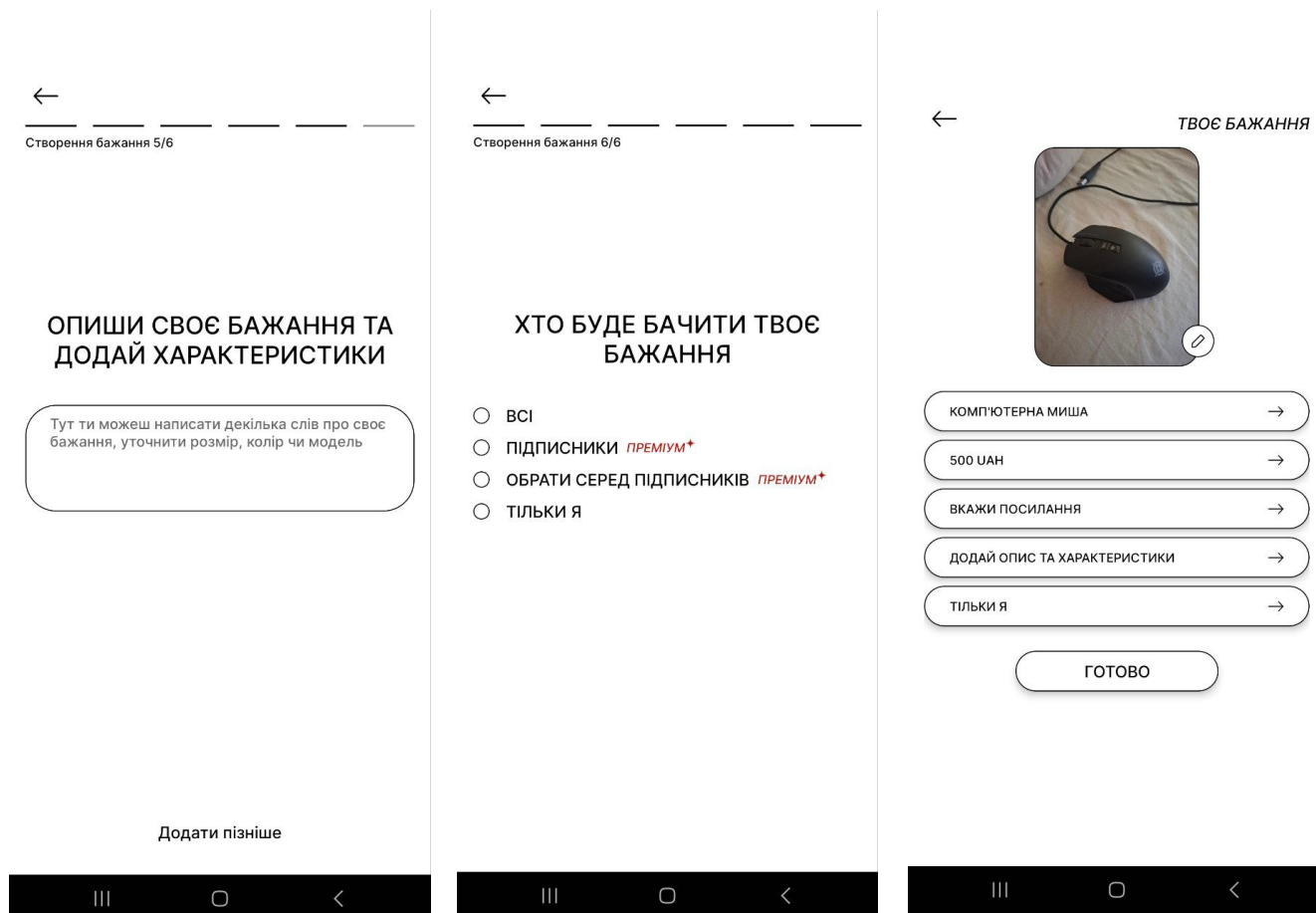


Рис. 5.7, аркуш 2

Окремою перевагою цього блоку є підтримка мультимедіа. Користувач може вибрати фото або відео, обрізати матеріал, налаштувати обкладинку для відео та перевірити фінальний вигляд бажання перед публікацією. Така побудова сценарію зменшує кількість помилок і наближає процес створення контенту до звичних патернів сучасних мобільних застосунків.

5.2.8. Профіль користувача

На сторінці профілю користувач може переглядати особисті бажання та чернетки і має можливість поділитись своїм профілем.



Рис. 5.8 – Сторінка профілю користувача

5.2.9. Сторінка налаштувань

Сторінка налаштувань містить зручно розташовані кнопки, що ведуть до додаткових можливостей додатку та його параметрів.

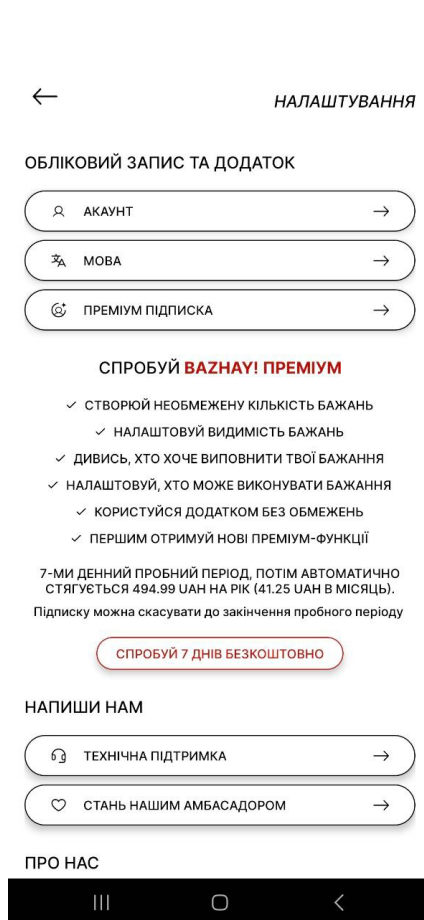


Рис. 5.9 – Сторінка налаштувань додатку

5.2.10. Сторінка редагування профілю

Сторінка редагування профілю дає змогу вказати адресу для отримання подарунків і змінити поля, заповнені під час реєстрації.

Рис. 5.10 – Сторінка редагування профілю

5.2.11. Сторінки налаштування мови та преміум підписки

Сторінки налаштування мови та преміум-підписки надають інтерфейси для зміни мови, а також для перегляду поточного стану підписки й доступних пропозицій.

МОВА

УКРАЇНСЬКА ✓

АНГЛІЙСЬКА

ПРЕМІУМ ПІДПИСКА

ДЯКУЄМО, ЩО КОРИСТУЄШСЯ ПРЕМІУМ-ВЕРСІЄЮ BAZHAY!

ПЕРЕВАГИ ПРЕМІУМ-ПІДПИСКИ:

- ✓ СТВОРИТИ НЕОБМЕЖЕНУ КІЛЬКІСТЬ БАЖАНЬ
- ✓ НАЛАШТУВАТИ ВИДИМІСТЬ БАЖАНЬ
- ✓ ДИВИСЬ, ХТО ХОЧЕ ВИПОВНИТИ ТВОЇ БАЖАННЯ
- ✓ НАЛАШТУВАТИ, ХТО МОЖЕ ВИКОНУВАТИ БАЖАННЯ
- ✓ КОРИСТУЙСЯ ДОДАТКОМ БЕЗ ОБМЕЖЕНЬ
- ✓ ПЕРШИМ ОТРИМУЙ НОВІ ПРЕМІУМ-ФУНКЦІЇ

У тебе пробний 7-ми денний період преміум-версії BAZHAY! По закінченню пробного періоду 16.05.2025 буде автоматично оформлена річна підписка у відсутності відміни підписки як мінімум за 24 години до цієї дати. річна підписка коштує 494.99 UAH (41.25 UAH на місяць).

ПОШИРЕНІ ПИТАННЯ

ТЕХНІЧНА ПІДТРИМКА

Відновити покупки Правила та умови

ПРЕМІУМ ПІДПИСКА

ПЕРЕВАГИ ПРЕМІУМ-ПІДПИСКИ:

- ✓ СТВОРИТИ НЕОБМЕЖЕНУ КІЛЬКІСТЬ БАЖАНЬ
- ✓ НАЛАШТУВАТИ ВИДИМІСТЬ БАЖАНЬ
- ✓ ДИВИСЬ, ХТО ХОЧЕ ВИПОВНИТИ ТВОЇ БАЖАННЯ
- ✓ НАЛАШТУВАТИ, ХТО МОЖЕ ВИКОНУВАТИ БАЖАННЯ
- ✓ КОРИСТУЙСЯ ДОДАТКОМ БЕЗ ОБМЕЖЕНЬ
- ✓ ПЕРШИМ ОТРИМУЙ НОВІ ПРЕМІУМ-ФУНКЦІЇ

НА РІК 494.99 UAH
41.25 UAH в місяць

МІСЯЦЬ 149.99 UAH

Відновити покупки Правила та умови

Рис. 5.11 – Сторінки налаштування мови та преміум підписки

5.2.12. Сторінки поширених питань та технічної підтримки.

Сторінки поширених питань та технічної підтримки надають інформацію про поширені питання, що можуть виникнути в користувача, а також інтерфейс для зв'язку з менеджером додатку, що дає можливість повідомити про проблему, або отримати відповідь на запитання.

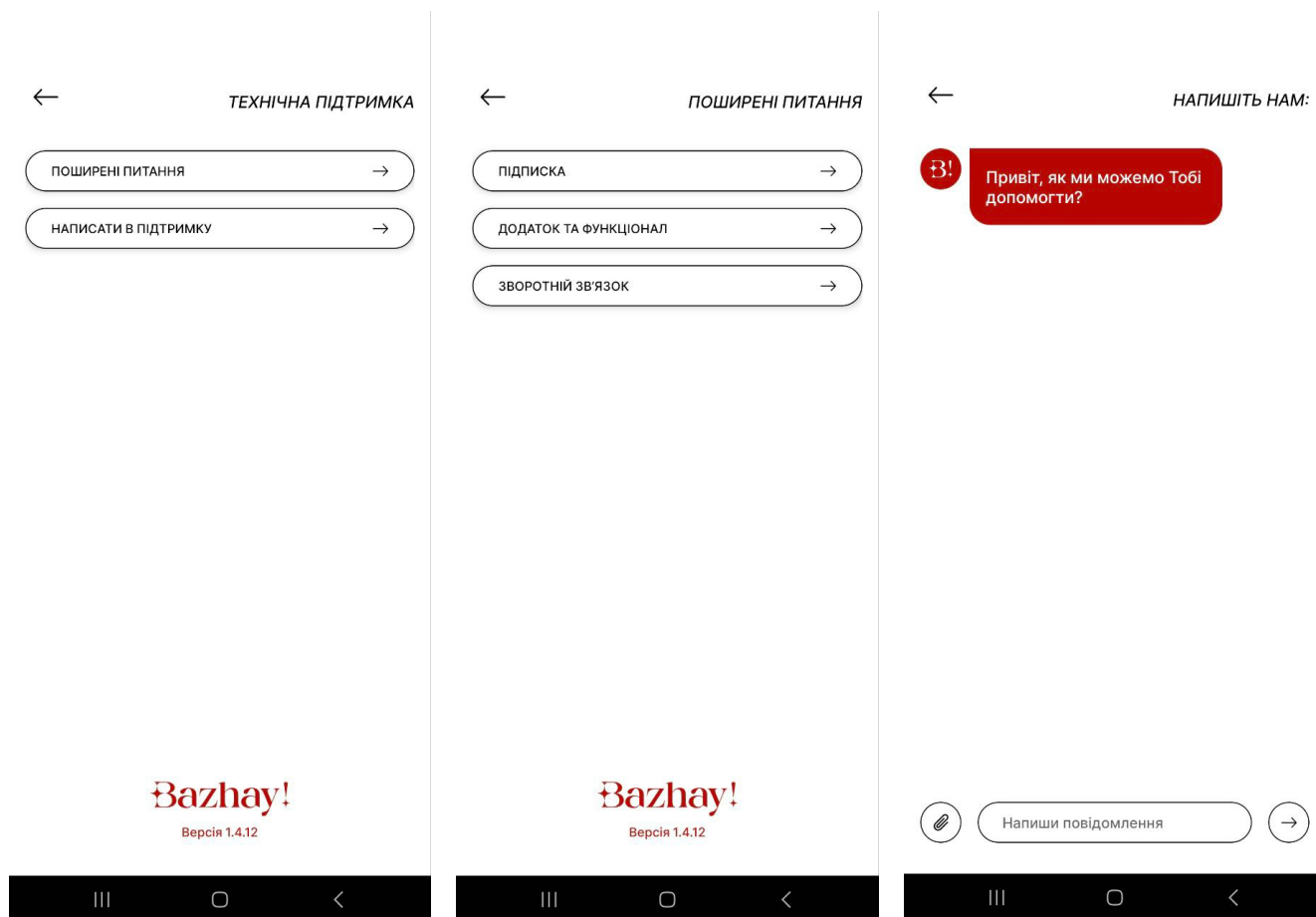


Рис. 5.12 – Сторінки технічної підтримки

Інтерфейс технічної підтримки дає змогу не лише поставити запитання, а й прикріпити файл, скриншот або відео з описом проблеми. Це суттєво пришвидшує аналіз помилок і дає змогу отримувати більш якісний зворотний зв'язок від користувачів.

5.2.13. Сторінка видалення профілю

Окремо реалізована сторінка для видалення профілю, щоб уникнути можливих випадкових дій користувача.

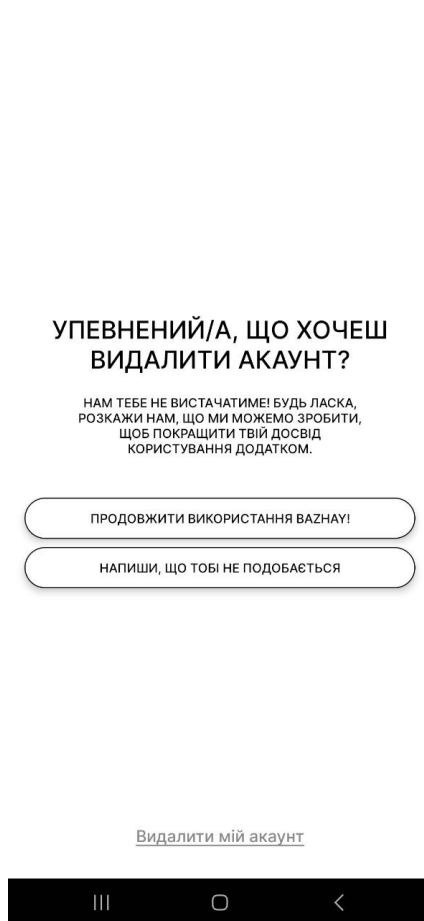


Рис. 5.13 – Сторінка видалення профілю

5.3. Додаткові елементи

Анімація логотипу при запуску додатку.

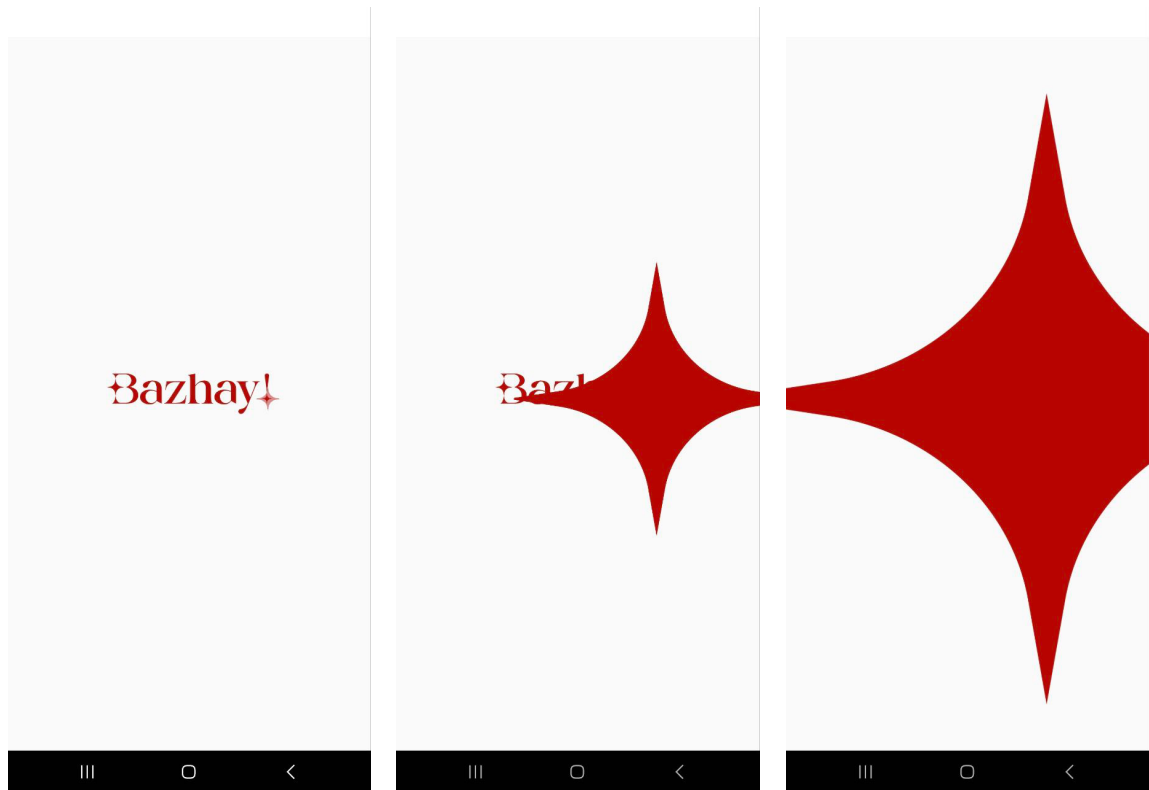


Рис. 5.14 – Анімація логотипу

Поп-ап повідомлення для підтверджень, помилок, успішних дій.

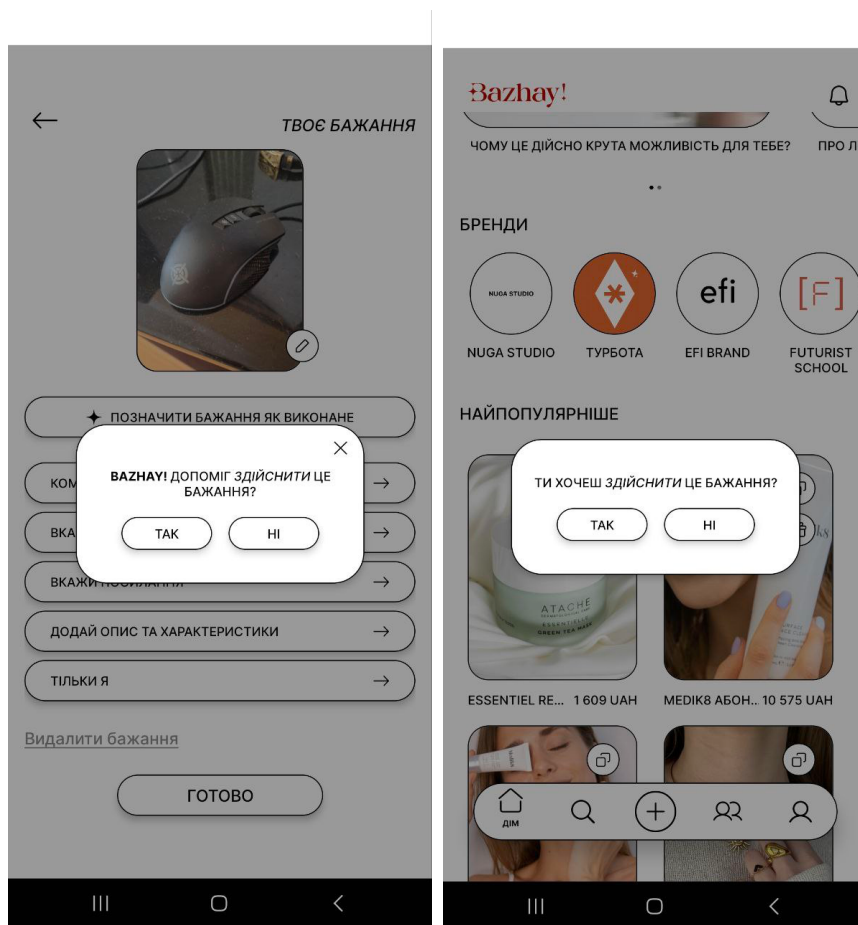


Рис. 5.15 – Поп-ап повідомлення

До додаткових інтерактивних елементів також належать push-сповіщення та deep links. Вони дозволяють повертати користувача безпосередньо до потрібного сценарію взаємодії: перегляду новини, профілю, конкретного бажання або повідомлення від служби підтримки.

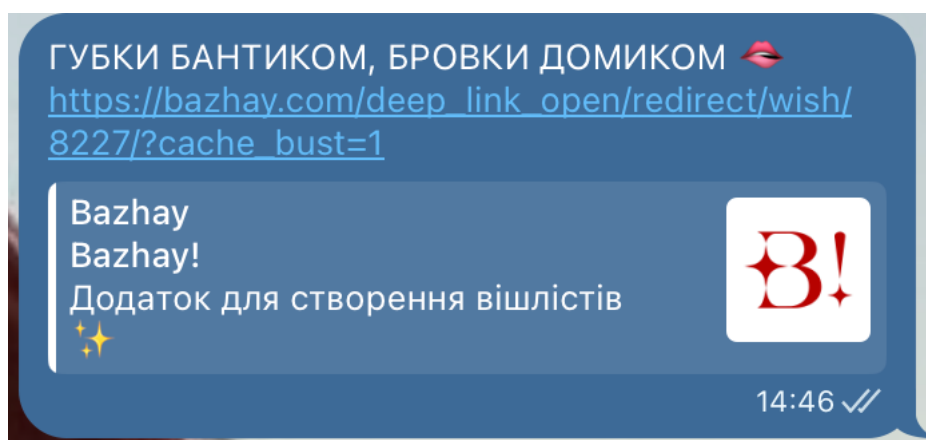


Рис. 5.16 – Повідомлення deep link в месенджері

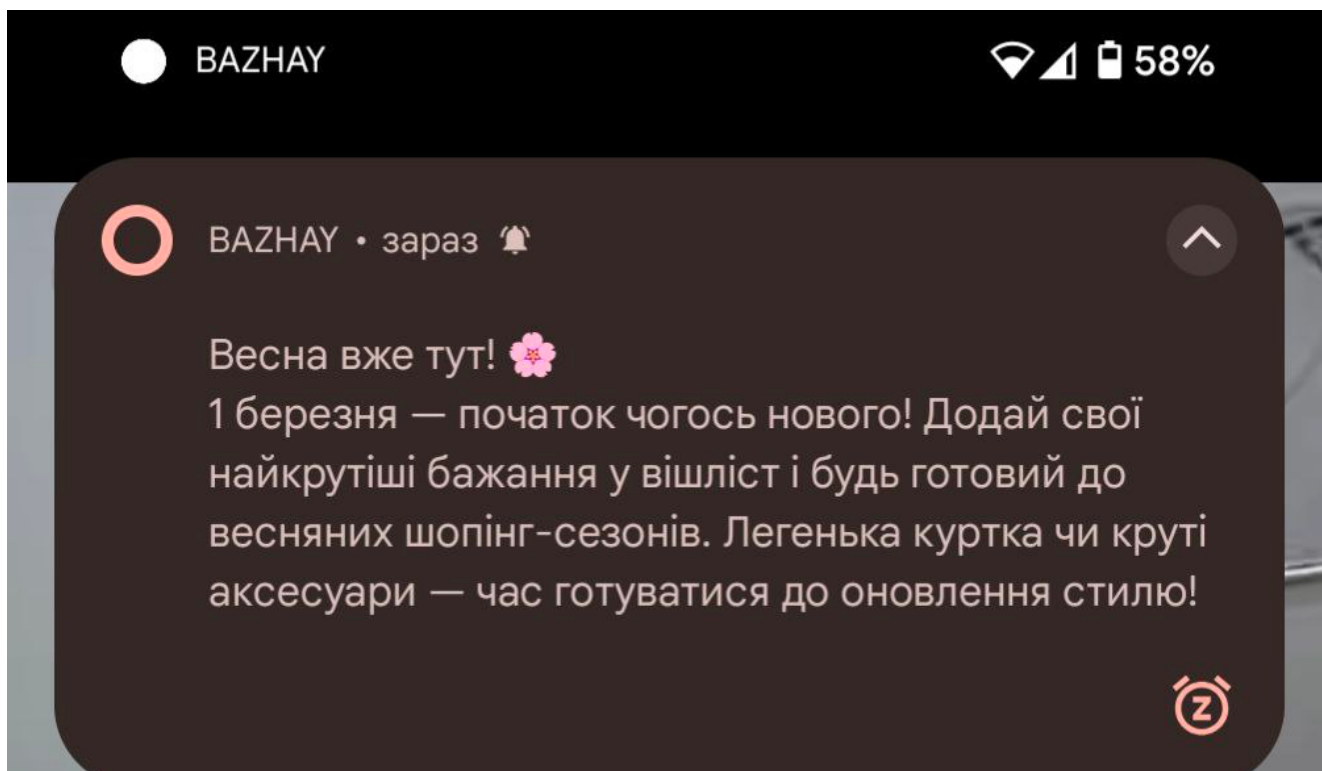


Рис. 5.17 – Push-сповіщення

5.4. Платформи

Додаток реалізований кросплатформено – підтримується Android та iOS. Він вже доступний для завантаження в **Google Play** [18] та **App Store** [19].

5.4.1. Конфігурація платформ

Платформозалежні параметри централізовано описано у конфігурації **app.json**. Саме тут визначаються ідентифікатори пакетів для iOS та Android, пов'язані домени для deep links, підтримка входу через Apple, підключення Firebase-конфігурацій, дозволи на використання камери, галереї, доступу до медіафайлів, білінгу та інших системних можливостей. Окремо зазначено підтримку планшетів на iOS, що підтверджує орієнтацію проєкту на ширший спектр пристроїв.

Конфігурація також містить перелік Ехро-плагінів, через які підключаються локалізація, in-app purchase, Apple Authentication, Firebase, налаштування SDK-рівнів Android та доступ до медіатеки. Такий підхід дозволяє тримати критичні параметри середовища в одному місці і відтворювати однакову конфігурацію під час локальної розробки, тестування та побудови релізних збірок.

5.4.2. Публікація та оновлення

Для побудови та розповсюдження мобільного застосунку використовується екосистема Ехро та ЕАС. Вона спрощує підготовку білдів для App Store і Google Play, дозволяє вести окремі номери версій для iOS та Android, а також забезпечує керований процес оновлення застосунку перед публікацією в маркетплейсах.

Окремою перевагою є підтримка ОТА-оновлень через ехро-updates. Це дає змогу доставляти частину змін у JavaScript-код і інтерфейсі без повного повторного проходження життєвого циклу релізу в магазинах застосунків. У результаті маємо гнучкіший процес супроводу продукту, а користувачі швидше одержують виправлення помилок та дрібні покращення.

6. ОТРИМАНІ РЕЗУЛЬТАТИ ТА МОЖЛИВОСТІ ДЛЯ ПОДАЛЬШОГО РОЗВИТКУ

6.1. Отримані результати

У результаті виконання бакалаврської роботи було розроблено та впроваджено клієнтську частину мобільного додатку «**BAZHAY!**», який реалізує повноцінний функціонал для створення, перегляду, резервування та дарування бажань.

Ключові результати:

- Створено зручний і стабільний інтерфейс користувача з урахуванням досвіду UI/UX.
- Реалізовано авторизацію та аутентифікацію користувачів із підтвердженням через код.
- Розроблено систему створення бажань із підтримкою універсального парсера посилань.
- Забезпечено механізм перегляду та резервування бажань інших користувачів.
- Оптимізовано архітектуру клієнтської частини з використанням сучасного стеку технологій (React Native [1], Expo [2], Firebase [3]).
- Виконано ручне тестування додатку для виявлення та усунення помилок.
- Додаток опубліковано у Google Play [18] та App Store [19], де активно використовується цільовою аудиторією.

Окрему практичну цінність мають реалізовані механізми багатомовності, push-сповіщень, deep linking, інтеграції преміум-підписки, розширеної обробки фото і відео та каналу технічної підтримки. Саме ці підсистеми наближають розроблений застосунок до рівня повноцінного комерційного продукту, готового до реальної експлуатації.

Додатково в роботі реалізовано універсальний кешувальний шар, аналітику поведінки користувачів, моніторинг стабільності та платформену конфігурацію

для публікації застосунку. У сукупності ці рішення забезпечують не лише функціональність, а й готовність продукту до тривалого супроводу та масштабування.

6.2. Новизна роботи

Новизна роботи полягає у поєднанні соціального механізму бажань, інтеграції парсера посилок для швидкого створення бажань, інтуїтивної системи резервування та можливості копіювання бажань інших користувачів.

6.3. Перспективи подальшого розвитку

Розроблений проєкт має потенціал для масштабування та доповнення новими функціями, зокрема:

- Додавання **чатів між користувачами** для обговорення подарунків або планування спільних подарунків.
- Впровадження **штучного інтелекту** для рекомендації подарунків на основі інтересів користувача.
- Підключення **оплати в додатку** для реалізації бажань прямо з мобільного пристрою.
- Створення **веб-версії** платформи.
- Впровадження **системи рейтингів брендів** та розширення рекламних можливостей для бізнесу.

ВИСНОВКИ

У процесі розробки клієнтської частини мобільного додатку «BAZHAY!» було досліджено та реалізовано сучасні підходи до створення кросплатформеного програмного забезпечення з фокусом на зручність користувача, адаптивність інтерфейсу та надійність функціоналу. На основі виконаної роботи можна зробити такі висновки:

1. Актуальність проблеми дарування небажаних подарунків знайшла ефективне вирішення у вигляді цифрового продукту, що дозволяє користувачам формувати власні списки бажань і ділитися ними з оточенням. Це дозволяє уникнути невизначеності при виборі подарунків і знижує соціальну напругу під час свят.
2. Правильно обрана архітектура клієнтської частини та стек технологій (React Native [1] + Expo [2]) забезпечили ефективну розробку, легкість розгортання та подальшу підтримку застосунку на платформах Android та iOS.
3. Інтеграція парсера сайтів як засобу автоматичного створення бажань є технічно складним, але надзвичайно корисним рішенням, що значно покращує користувацький досвід.
4. Розробка інтерфейсу з урахуванням принципів UX/UI дозволила досягти інтуїтивності у взаємодії з додатком, що підтверджується позитивними відгуками користувачів після релізу.
5. Результати ручного тестування свідчать про достатній рівень стабільності та коректності реалізованого функціоналу в рамках заявлених вимог.
6. Практична реалізація додатку, його публікація в маркетплейсах і реальне використання цільовою аудиторією підтверджують не лише доцільність, а й життєздатність запропонованого рішення.

7. Проєкт має високий потенціал подальшого розвитку, включно з розширенням функціоналу, соціальними механіками та комерційними можливостями, що відкриває перспективи його масштабування.

Також, варто зазначити, що під час розробки програмної реалізації було використано інструмент штучного інтелекту ChatGPT (OpenAI) для пошуку релевантної інформації та програмне забезпечення Codex (OpenAI) для написання рутинних частин програмного коду.

Таким чином, бакалаврська робота досягла поставленої мети, реалізувавши повноцінний цифровий продукт із цінністю для реальних користувачів та потенціалом для бізнесового застосування.

Код розробленого мобільного застосунку знаходиться за покликанням:
<https://github.com/Vitya77/BAZHAY>

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. <https://reactnative.dev/> - офіційна документація фреймворку React Native.
2. <https://expo.dev/> - офіційний веб-сайт середовища Expo.
3. <https://firebase.google.com/> - офіційний веб-сайт Google Firebase.
4. <https://www.django-rest-framework.org/> - офіційний веб-сайт фреймворку Django REST.
5. <https://www.postgresql.org/> - офіційний веб-сайт PostgreSQL.
6. <https://redis.io/> - офіційний веб-сайт Redis.
7. <https://www.apple.com/app-store/> - платформа App Store.
8. <https://play.google.com/> - платформа Google Play.
9. <https://firebase.google.com/docs/analytics> - офіційна документація Firebase Analytics.
10. <https://reactnavigation.org/> - офіційна документація бібліотеки React Navigation.
11. <https://docs.expo.dev/versions/latest/sdk/async-storage/> - документація бібліотеки AsyncStorage.
12. <http://formik.org/docs/guides/react-native> - офіційна документація бібліотеки Formik в поєднанні з React Native.
13. <https://www.npmjs.com/package/yup> - бібліотека Yup.
14. <https://github.com/DylanVann/react-native-fast-image> - репозиторій бібліотеки React Native Fast Image.
15. <https://docs.swmansion.com/react-native-reanimated/> - документація бібліотеки React Native Reanimated.
16. <https://react.dev/reference/react/useContext> - документація по хуку useContext.
17. <https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest> - документація по XMLHttpRequest.

18. <https://play.google.com/store/apps/details?id=com.ivanfranco.BAZHAYRN&hl=uk> – додаток «BAZHAY!» на платформі Google Play.
19. <https://apps.apple.com/ua/app/bazhay/id6737126307> - додаток «BAZHAY!» на платформі App Store.